

DA5300: Data Structures For Data Science

Laboratory 7- Min Heap

Objectives

In this laboratory, you will implement a **Min-Heap** using an array-based representation. You will implement insertion, deletion, heapify-up, heapify-down, extract-min, and display the heap as a complete binary tree.

Exercise: Min-Heap

A Min-Heap is a complete binary tree in which every parent key is less than or equal to the keys of its children. If the heap is represented by an array A , then for an element at index i ,

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{left}(i) = 2i + 1, \quad \text{right}(i) = 2i + 2.$$

For every non-root element,

$$A[\text{parent}(i)] \leq A[i].$$

1. Array Representation

Use a Python list as the representation of the complete binary tree. Do **not** create explicit left and right child references.

For example,

```
[10, 20, 60, 40, 30]
```

represents

```
      10
     /  \
    20   60
   /  \
  40  30
```

2. MinHeap Class

Create a class named `MinHeap`. Implement at least:

```
insert(key)
delete(key)
extract_min()
peek()
heapify_up(index)
heapify_down(index)
is_empty()
__len__()
```

You may create additional helper methods.

3. Insertion and Heapify-Up

When inserting a key:

1. append the key to the end of the array;
2. compare it with its parent;
3. if the parent is larger, exchange the two;
4. continue upwards until the Min-Heap property is restored.

This process is called **heapify-up**.

4. Extract-Min and Heapify-Down

The minimum element is always at index 0. Implement `extract_min()` as follows:

1. save the root;
2. move the last element to index 0;
3. remove the last position;
4. restore the Min-Heap property using heapify-down;
5. return the saved root.

Calling `extract_min()` on an empty heap should raise an appropriate exception.

5. Delete an Arbitrary Key

Implement `delete(key)`. Remove one occurrence of the specified key.

A suitable procedure is:

1. find the index of the key;
2. replace it with the last element;
3. remove the last array element;
4. if the replacement is smaller than its parent, heapify-up;
5. otherwise heapify-down.

If the key does not exist, leave the heap unchanged.

6. Peek

Implement `peek()`, which returns the minimum without removing it. Calling it on an empty heap should raise an appropriate exception.

Input Format

The input is a text file.

- Odd-numbered lines contain keys to be **inserted**.
- Even-numbered lines contain keys to be **deleted**.
- Keys on each line are separated by commas.

- Operations are cumulative.

For example:

```
40 , 20 , 60 , 10 , 30
20
5 , 25
40
```

means insert the first five keys, delete 20, insert 5 and 25, and finally delete 40.

Output Format

The program must produce:

```
heap.txt
sorted.txt
tree.txt
```

There must be exactly one line in `heap.txt` and `sorted.txt` for every input line.

`heap.txt`

Store the array representation of the heap after every input line. Keys are comma-separated.

`sorted.txt`

For every input line, output the keys in increasing order. The sorted sequence must be obtained by repeatedly performing `extract_min()` on a **copy** of the current heap. The current heap must not be destroyed.

`tree.txt`

Store one tree for every input line. Consecutive trees are separated by a blank line.

The last level must start at the beginning of the line. Nodes on the last level must be separated by exactly one whitespace. Students should use whitespace intelligently on the upper levels to show the parent-child relationship.

For example:

```
      10
    20      60
40  30
```

The exact upper-level indentation is flexible, but the tree structure must be correct. Missing children must not be represented by dummy values.

Command-Line Execution

The program must be named `heap.py`.

Run:

```
python heap.py test1.txt heap.txt sorted.txt tree.txt
```

Repeat using `test2.txt` and `test3.txt`.

Do not hard-code file names.

The Python `heapq` module must **not** be used.

Test Cases

Test Case 1

```
40 , 20 , 60 , 10 , 30
20
5 , 25
40
```

Test Case 2

```
50 , 40 , 30 , 20 , 10
50 , 10
5 , 15 , 35
20 , 5
```

Test Case 3

```
30 , 10 , 20 , 5 , 40 , 15 , 25 , 2
10 , 40
1 , 35 , 50
15 , 2 , 30
```

Evaluation

The supplied program `heap_evaluate.py` independently constructs the expected Min-Heap and checks:

1. the number of output lines;
2. the heap array after every input line;
3. the Min-Heap property;
4. the sorted output after every input line;
5. the number of trees in `tree.txt`;
6. the node sequence at every level of each displayed tree.

Run:

```
python heap_evaluate.py test1.txt heap.txt sorted.txt tree.txt
```

and similarly for the other two test cases.

The evaluator prints **PASS** if all checks succeed; otherwise it prints **FAIL** and identifies the failed component.

Whitespace used for positioning upper levels of `tree.txt` is ignored by the evaluator. The level structure and node values are checked.